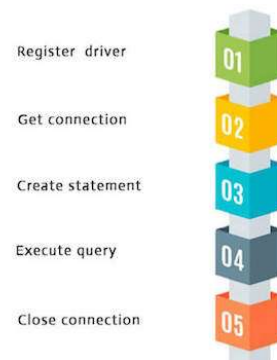


## 3. Java Database Connectivity with 5 Steps

There are 5 steps to connect any java application with the database using JDBC. These steps are as follows:

- Register the Driver class
- Create connection
- Create statement
- Execute queries
- Close connection

### Java Database Connectivity



### 3.1 Register the driver class

The **forName()** method of Class class is used to register the driver class. This method is used to dynamically load the driver class.

#### Syntax of forName() method

```
public static void forName(String className)throws ClassNotFoundException
```

**Note:** Since JDBC 4.0, explicitly registering the driver is optional. We just need to put vender's Jar in the classpath, and then JDBC driver manager can detect and load the driver automatically.

#### Example to register the Oracle Driver class

Here, Java program is loading oracle driver to establish database connection.

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

### 3.2 Create the connection object

The getConnection() method of DriverManager class is used to establish connection with the database.

#### Syntax of getConnection() method

```
public static Connection getConnection(String url) throws SQLException  
public static Connection getConnection(String url,String name,String password)  
throws SQLException
```

#### Example to establish connection with the Oracle database

```
Connection con=DriverManager.getConnection( "jdbc:oracle:thin:@localhost:1521:x  
e","system","password");
```

### 3.3 Create the Statement object

The createStatement() method of Connection interface is used to create statement. The object of statement is responsible to execute queries with the database.

**Syntax of createStatement() method**

public Statement createStatement() throws SQLException

**Example to create the statement object**

```
Statement stmt=con.createStatement();
```

### 3.4 Execute the query

The executeQuery() method of Statement interface is used to execute queries to the database. This method returns the object of ResultSet that can be used to get all the records of a table.

**Syntax of executeQuery() method**

public ResultSet executeQuery(String sql) throws SQLException

**Example to execute query**

```
ResultSet rs = stmt.executeQuery("select * from emp");
while(rs.next()) {
    System.out.println(rs.getInt(1)+" "+rs.getString(2));
}
```

### 3.5 Close the connection object

By closing connection object statement and ResultSet will be closed automatically. The close() method of Connection interface is used to close the connection.

**Syntax of close() method**

public void close() throws SQLException

**Example to close connection**

```
con.close();
```

**Note:** Since Java 7, JDBC has ability to use try-with-resources statement to automatically close resources of type Connection, ResultSet, and Statement.